View Source Edit Edit Source Tools

PAPI (People API)

Primary Contact 017777087762 (AWS account) *How do I change this value?*
Last modified 2 months ago by sigv4-017777087762.

PAPI (People API)

Overview

The PAPI (People API) Client provides integration with Amazon's People API Service for searching and retrieving employee information. It uses CloudAuth for authentication via AAA (Amazon Authentication and Authorization) and is accessed through SuperStar service discovery.

Official Documentation: PAPI v2 API Documentation

Package Location

```
packages/clients/src/papi-client/
├── client.ts          # PapiClient implementation
└── types.ts           # TypeScript types and interfaces
```

Authentication & Access

CloudAuth via AAA

The PAPI client authenticates using CloudAuth with AAA credentials:

- **Service Name**: `PeopleAPIService`
- **Authentication**: CloudAuth with TransitiveAuth support
- **Access Method**: SuperStar service discovery
- **Endpoint**: Configured via `PAPI_ENDPOINT` environment variable

SuperStar Integration

SuperStar provides the PAPI endpoint URL based on the deployment environment:

- **Beta**: `papi-beta.amazon.com`
- **Gamma**: `papi-gamma.amazon.com`
- **Prod**: `papi.amazon.com`

The endpoint is injected into the Lambda environment via CDK configuration.

Client Architecture

Singleton Pattern

The PAPI client uses a singleton pattern to ensure a single instance across the application:

```
const client = PapiClient.getInstance()
```

Base Class

Extends `BaseCloudAuthARestClient` which provides:

- CloudAuth token management
- Automatic retries (default: 3 attempts)
- Timeout handling (default: 30 seconds)
- TransitiveAuth support for human identity propagation

API Reference

peopleSearchV3

Search for employees using PAPI v3 search API with complex query support.

```
public async peopleSearchV3(
  request: PapiV3SearchRequest,
  humanIdentity?: string
): Promise<PapiV3SearchResponse>
```

Parameters

request: `PapiV3SearchRequest`

- `query`: Search query structure (required)
    - `occurrence`: `'MUST' | 'SHOULD' | 'MUST_NOT'`
    - `minimumShouldMatch`: Minimum number of SHOULD clauses that must match
    - `nestedSearchQueries`: Nested query structures for complex searches
    - `expressions`: Array of search expressions (term, terms, range, fuzzy, wildcard, template)
- `nextToken`: Pagination token for retrieving next page (optional)
- `size`: Number of results to return (optional, default: 10)
- `sortBy`: Sort field (optional)
    - `'FIRST_NAME'`
    - `'FULL_NAME'`
    - `'LAST_NAME'`
    - `'LOGIN'`
    - `'WEIGHT_THEN_FIRST_NAME'`
    - `'WEIGHT_THEN_FULL_NAME'`
    - `'WEIGHT_THEN_LAST_NAME'`
    - `'WEIGHT_THEN_LOGIN'`
- `sortOrder`: `'ASC' | 'DESC'` (optional)

humanIdentity: `string` (optional)

- IDMS person ID for TransitiveAuth initiation
- Used to propagate user identity through the call chain

Returns

`Promise<PapiV3SearchResponse>`

- `records`: Array of `EmployeeV3` objects
- `nextToken`: Token for next page (null if no more results)

Example Usage

```
import { PapiClient } from '@amzn/hiring-hub-clients'

const client = PapiClient.getInstance()

// Search by name using NAME template
const response = await client.peopleSearchV3(
	{
		query: {
			occurrence: 'MUST',
			expressions: [
				{
					template: {
						type: 'NAME',
						params: {
							value: 'John Doe'
						}
					}
				}
			]
		},
		sortBy: 'WEIGHT_THEN_LOGIN',
		size: 10
	},
	'user123'
)

console.log(`Found ${response.records.length} employees`)
response.records.forEach(employee => {
	console.log(
		`${employee.basicInfo?.firstName} ${employee.basicInfo?.lastName} (${employee.basicInfo?.login})`
	)
})
```

Search Query Expressions

PAPI v3 supports multiple expression types for flexible searching. All expressions are based on the PAPI v2 API documentation.

Template Expression (Recommended)

The NAME template is the recommended way to search for people by name or login:

```
{
  template: {
    type: 'NAME',
    params: {
      value: 'John Doe'  // Searches firstName, lastName, and login
    }
  }
}
```

Benefits:

- Searches across multiple fields automatically (firstName, lastName, login)
- Provides relevance scoring
- Handles fuzzy matching
- No need to construct complex nested queries

Term Expression

Search for exact matches on a single field:

```
{
  term: {
    attribute: 'login',
    value: 'jdoe',
    boost: 2.0  // Optional relevance boost
  }
}
```

Terms Expression

Search for multiple exact matches on a single field:

```
{
  terms: {
    attribute: 'jobLevel',
    value: ['L5', 'L6', 'L7'],
    boost: 1.5
  }
}
```

Range Expression

Search for values within a range:

```
{
  range: {
    attribute: 'jobLevel',
    gte: 'L5',  // Greater than or equal
    lte: 'L7',  // Less than or equal
    boost: 1.0
  }
}
```

Fuzzy Expression

Search with fuzzy matching (typo tolerance):

```
{
  fuzzy: {
    attribute: 'firstName',
    value: 'Jhon',  // Will match "John"
    boost: 0.8
  }
}
```

Wildcard Expression

Search with wildcard patterns:

```
{
  wildcard: {
    attribute: 'login',
    value: 'j*doe',  // Matches jdoe, johndoe, etc.
    boost: 1.0
  }
}
```

Complex Query Examples

Search by Name or Login

```
const response = await client.peopleSearchV3({
	query: {
		occurrence: 'MUST',
		expressions: [
			{
				template: {
					type: 'NAME',
					params: { value: 'John' }
				}
			}
		]
	},
	size: 20
})
```

Search with Multiple Criteria

```
const response = await client.peopleSearchV3({
	query: {
		occurrence: 'MUST',
		nestedSearchQueries: [
			{
				occurrence: 'MUST',
				expressions: [
					{
						template: {
							type: 'NAME',
							params: { value: 'John' }
						}
					}
				]
			},
			{
				occurrence: 'MUST',
				expressions: [
					{
						term: {
							attribute: 'jobLevel',
							value: 'L6'
						}
					}
				]
			}
		]
	},
	sortBy: 'WEIGHT_THEN_FULL_NAME',
	size: 10
})
```

Search with SHOULD Clauses

```
const response = await client.peopleSearchV3({
	query: {
		occurrence: 'SHOULD',
		minimumShouldMatch: 1,
		expressions: [
			{
				term: {
					attribute: 'costCenterName',
					value: 'Engineering'
				}
			},
			{
				term: {
					attribute: 'costCenterName',
					value: 'Product'
				}
			}
		]
	},
	size: 50
})
```

BFF Integration

get-employee-search Operation

The BFF provides a simplified operation that wraps the PAPI client:

```
// packages/bff-core/src/operations/get-employee-search/get-employee-search.ts

export async function getEmployeeSearch(
	params: GetEmployeeSearchParams
): Promise<GetEmployeeSearchResult> {
	const { query, pageSize = 10, humanIdentity } = params

	const client = PapiClient.getInstance()

	const response = await client.peopleSearchV3(
		{
			query: {
				occurrence: 'MUST',
				expressions: [
					{
						template: {
							type: 'NAME',
							params: { value: query }
						}
					}
				]
			},
			sortBy: 'WEIGHT_THEN_LOGIN',
			size: pageSize
		},
		humanIdentity
	)

	// Transform to simplified Employee type
	const employees = response.records
		.filter(record => record.basicInfo?.login)
		.map(record => ({
			username: record.basicInfo?.login?.toLowerCase() ?? '',
			fullName:
				`${record.basicInfo?.firstName} ${record.basicInfo?.lastName}`.trim(),
			businessTitle: record.basicInfo?.businessTitle,
			level: record.job?.jobLevel,
			departmentName: record.basicInfo?.costCenterName,
			departmentId: record.basicInfo?.costCenterId,
			city: record.workLocation?.city,
			country: record.workLocation?.country,
			employeeId: record.basicInfo?.personId
		}))

	return { employees }
}
```

API Endpoint

The BFF exposes this operation as:

```
GET /apis/AtoZHiringHubBFF/employee-search?query={query}&pageSize={pageSize}
```

See BFF Core Documentation for more details on BFF operations.

Error Handling

Client-Level Errors

The PAPI client handles errors at multiple levels:

1. **Network Errors**: Automatic retry with exponential backoff (3 attempts)
2. **Timeout Errors**: 30-second timeout per request
3. **Authentication Errors**: CloudAuth token refresh and retry
4. **Service Errors**: Logged and re-thrown with context

```
try {
	const response = await client.peopleSearchV3(request, humanIdentity)
} catch (error) {
	console.error('Error searching for employees with v3 API:', error)
	throw new Error('Error searching for employees with v3 API')
}
```

BFF-Level Error Handling

The BFF operation provides graceful degradation:

```
try {
	const response = await client.peopleSearchV3(request, humanIdentity)
	return { employees: transformedEmployees }
} catch (error) {
	console.error('Failed to search for people', { error })
	// Return empty array instead of throwing
	return { employees: [] }
}
```

This ensures the UI doesn't break if PAPI is unavailable.

Configuration

Environment Variables

Required environment variables:

```
# PAPI endpoint (injected by CDK)
PAPI_ENDPOINT=papi.amazon.com

# CloudAuth configuration (inherited from base client)
AWS_REGION=us-east-1
```

CDK Configuration

The PAPI endpoint is configured in the CDK stack:

```
// packages/cdk/backend/backend-api.ts

const papiEndpoint = getSuperStarEndpoint('PeopleAPIService', stage)

lambdaFunction.addEnvironment('PAPI_ENDPOINT', papiEndpoint)
```

Related Documentation

- PAPI v2 API Documentation - Official PAPI API reference
- PAPI Schema - Detailed employee data structure
- Employee Search Component - Frontend component using PAPI
- BFF Core - BFF operations including employee search
- CloudAuth Documentation - CloudAuth authentication

Tags: [+]

Information Comments (0) Attachments (0) History

Pages that transclude this page:	No pages transclude this page
MVP Replication Status:	More information about MVP replication can be found here.
	Instructions for accessing Wiki page view data can be found here.